

FatPTE - Expanding Page Table Entries for Security

Lukas Lamster¹ Martin Unterguggenberger¹ Moritz Waser¹ David Schrammel² Stefan Mangard¹

¹Graz University of Technology | ²RIVOS, Inc.

ARES 2025

> isec.tugraz.at

- **Page Table Entries hold security-critical metadata**
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

- introduce FatPTE
 - 📄 x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata
- perform a case study
 - 🔍 Investigate 10 commercial and academic countermeasures & requirements
- implement and evaluate x86 and RISC-V prototypes
 - ⚙️ x86 gem5 prototype and RISC-V FPGA implementation
 - 📊 1.34 % overhead for x86 and 1.99 % for RISC-V

❓ Problem Statement - Contributions

- Page Table Entries hold security-critical metadata
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

- introduce FatPTE
 - 📄 x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata
- perform a case study
 - 🔍 Investigate 10 commercial and academic countermeasures & requirements
- implement and evaluate x86 and RISC-V prototypes
 - ⚙️ x86 gem5 prototype and RISC-V FPGA implementation
 - 📊 1.34 % overhead for x86 and 1.99 % for RISC-V

❓ Problem Statement - Contributions

- Page Table Entries hold security-critical metadata
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

- introduce FatPTE
 -  x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata
- perform a case study
 -  Investigate 10 commercial and academic countermeasures & requirements
- implement and evaluate x86 and RISC-V prototypes
 -  x86 gem5 prototype and RISC-V FPGA implementation
 -  1.34 % overhead for x86 and 1.99 % for RISC-V

❓ Problem Statement - Contributions

- Page Table Entries hold security-critical metadata
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

- **introduce FatPTE**
 - 📄 **x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata**
- **perform a case study**
 - 🔍 Investigate **10 commercial and academic countermeasures & requirements**
- **implement and evaluate x86 and RISC-V prototypes**
 - ⚙️ **x86 gem5 prototype and RISC-V FPGA implementation**
 - 📊 **1.34 % overhead for x86 and 1.99 % for RISC-V**

❓ Problem Statement - Contributions

- Page Table Entries hold security-critical metadata
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

- introduce FatPTE
 - 📄 x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata
- perform a case study
 - 🔍 Investigate 10 commercial and academic countermeasures & requirements
- implement and evaluate x86 and RISC-V prototypes
 - ⚙️ x86 gem5 prototype and RISC-V FPGA implementation
 - 📊 1.34 % overhead for x86 and 1.99 % for RISC-V

❓ Problem Statement - Contributions

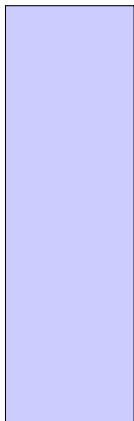
- Page Table Entries hold security-critical metadata
- On x86 and RISC-V: Barely any bits left
- New security features are hard to integrate

We ...

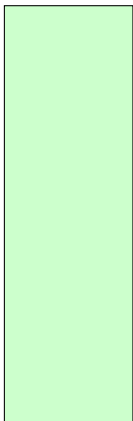
- introduce FatPTE
 - 📄 x86 and RISC-V hardware extension that enhances PTEs with dedicated metadata
- perform a case study
 - 🔍 Investigate 10 commercial and academic countermeasures & requirements
- implement and evaluate x86 and RISC-V prototypes
 - ⚙️ x86 gem5 prototype and RISC-V FPGA implementation
 - 📊 1.34 % overhead for x86 and 1.99 % for RISC-V

Background

Virtual
Memory

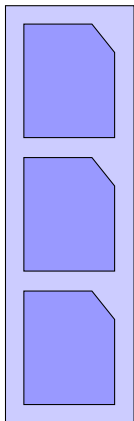


Physical
Memory

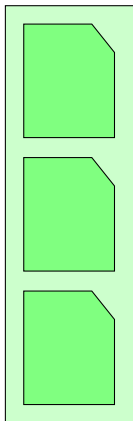


- **Virtual and physical** memory
- Both are divided into **pages**
- Minimum size on x86: **4kB**
- **VPNs** and **PPNs**
- **PT Hierarchy** maps VPNs to PPNs

Virtual
Memory

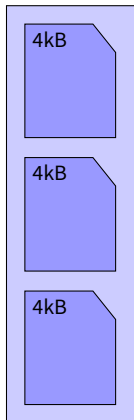


Physical
Memory

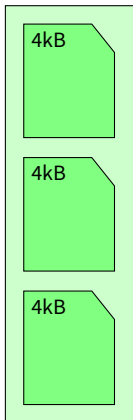


- **Virtual** and **physical** memory
- Both are divided into **pages**
- Minimum size on x86: **4kB**
- **VPNs** and **PPNs**
- **PT Hierarchy** maps VPNs to PPNs

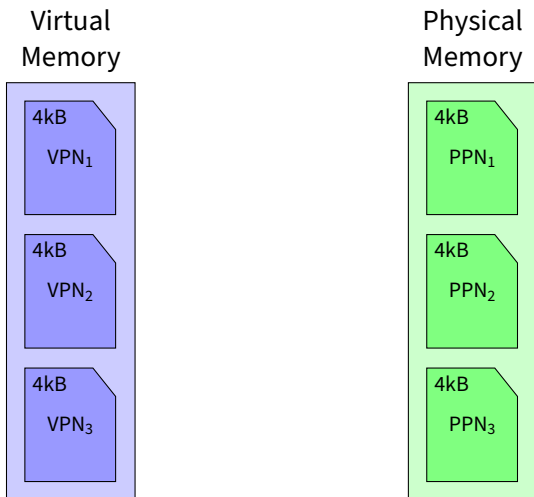
Virtual
Memory



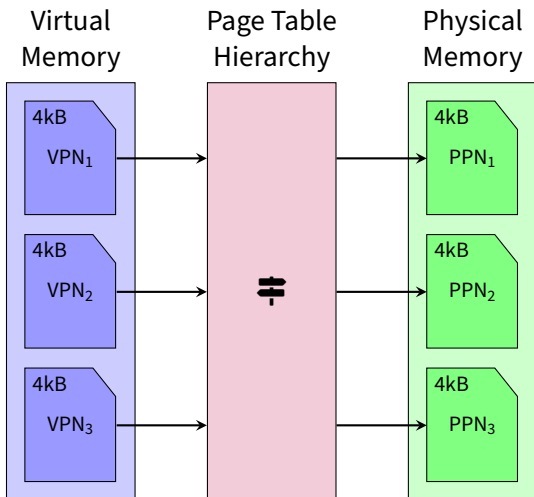
Physical
Memory



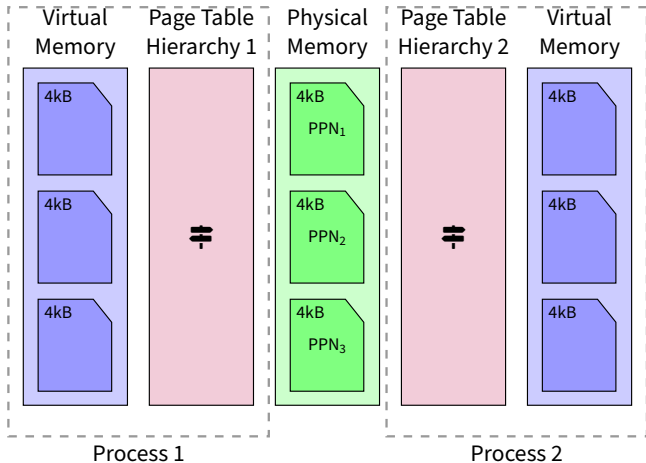
- **Virtual** and **physical** memory
- Both are divided into **pages**
- Minimum size on x86: **4kB**
- VPNs and PPNs
- PT Hierarchy maps VPNs to PPNs



- **Virtual** and **physical** memory
- Both are divided into **pages**
- Minimum size on x86: **4kB**
- **VPNs** and **PPNs**
- PT Hierarchy maps VPns to PPNs

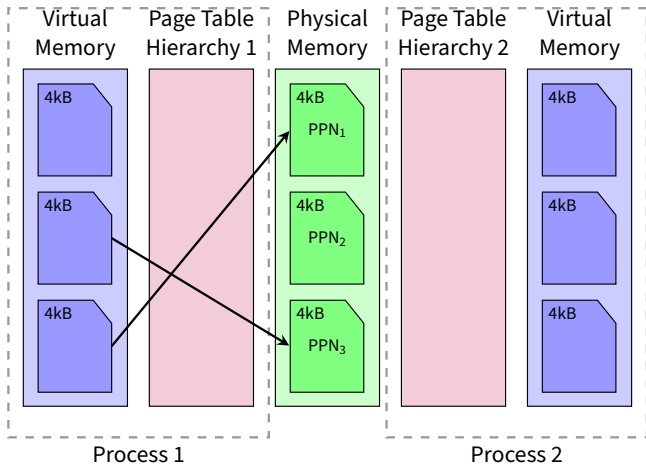


- **Virtual** and **physical** memory
- Both are divided into **pages**
- Minimum size on x86: **4kB**
- **VPNs** and **PPNs**
- **PT Hierarchy** maps VPns to PPNs



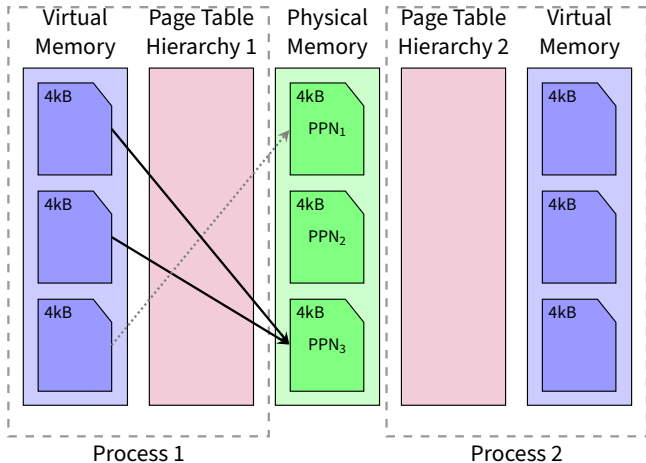
Benefits of paging

- Linear in VM $\neq$ Linear in PM
- Pages can **alias**
- Shared Memory** possible
- Each process has **exclusive view** of physical memory



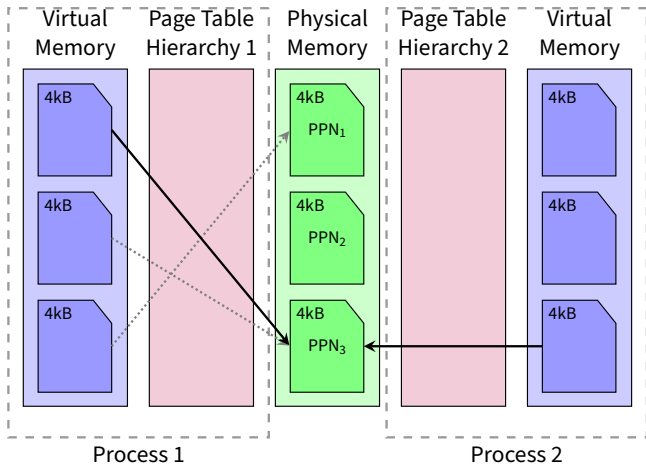
Benefits of paging

- Linear in VM $\neq$ Linear in PM
- Pages can alias
- Shared Memory possible
- Each process has **exclusive** view of physical memory



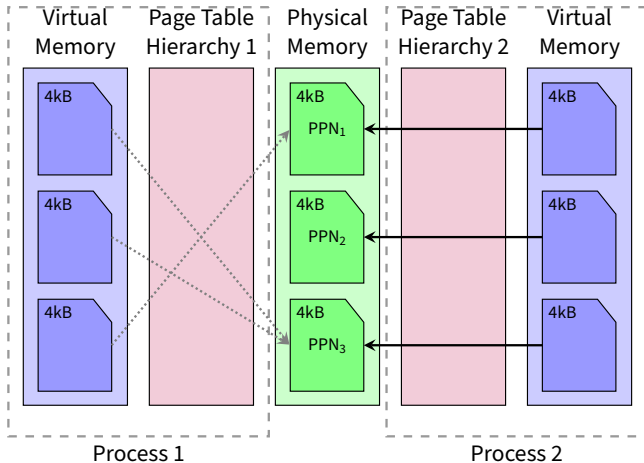
Benefits of paging

- Linear in VM $\neq$ Linear in PM
- Pages can **alias**
- Shared Memory possible
- Each process has **exclusive** view of physical memory



Benefits of paging

- Linear in VM $\neq$ Linear in PM
- Pages can **alias**
- **Shared Memory** possible
- Each process has **exclusive** view of physical memory



Benefits of paging

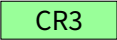
- Linear in VM $\neq$ Linear in PM
- Pages can **alias**
- **Shared Memory** possible
- Each process has **exclusive view** of physical memory

64-Bit Virtual Address



A horizontal grey bar representing a 64-bit virtual address. The text "Virtual Address" is centered within the bar.

Virtual Address



A small green rectangular box with a black border containing the text "CR3".

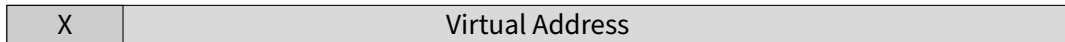
CR3

- **PTW** performs page table walk
- CR3 register points to root PPN
- 9 index bits per level
- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations

57-Bit

~~64-Bit~~ Virtual Address

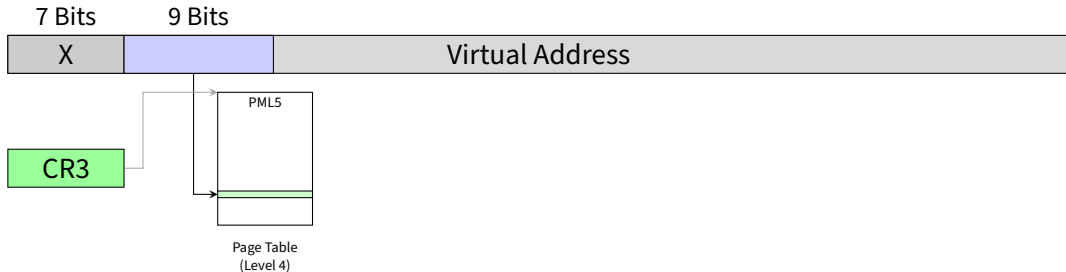
7 Bits



CR3

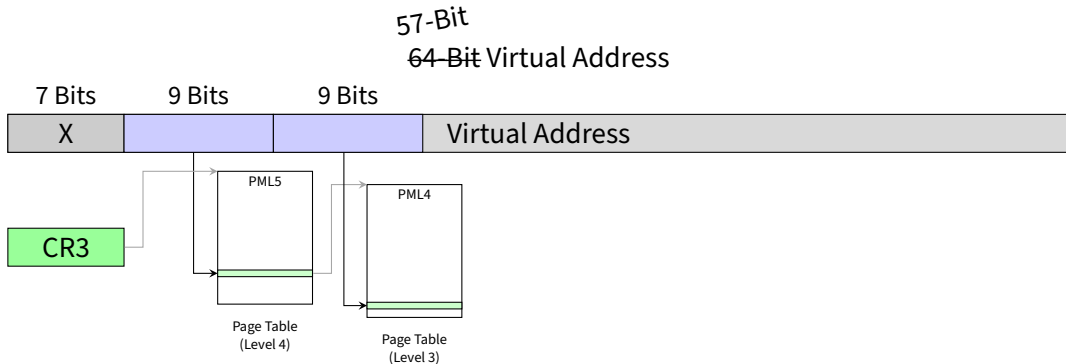
- **PTW** performs page table walk
- CR3 register points to root PPN
- 9 index bits per level
- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations

57-Bit
~~64-Bit~~ Virtual Address



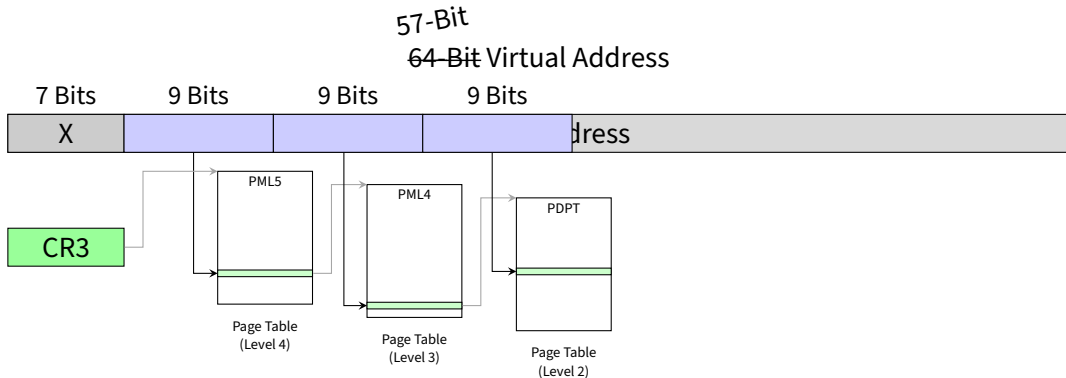
- **PTW** performs page table walk
- **CR3 register** points to **root PPN**
- **9 index bits** per level

- 12 index bits in leaf page
- **Translation Lookaside Buffer (TLB)** caches translations



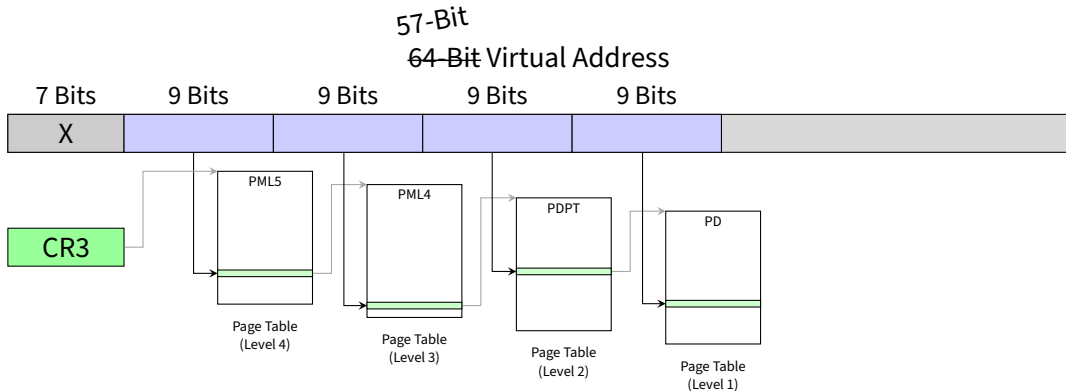
- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level

- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations



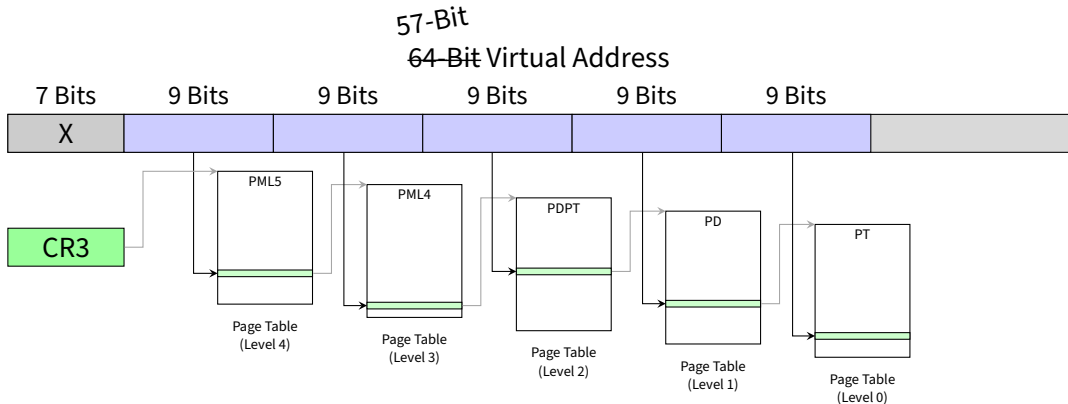
- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level

- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations



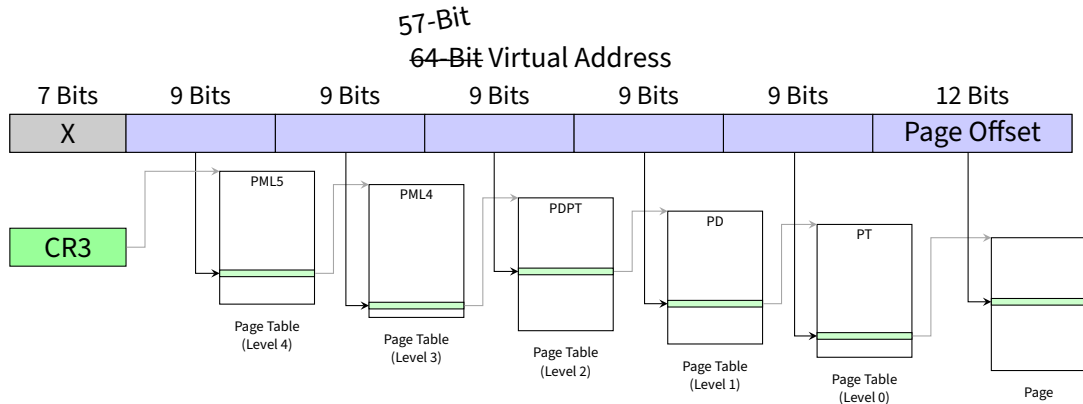
- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level

- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations



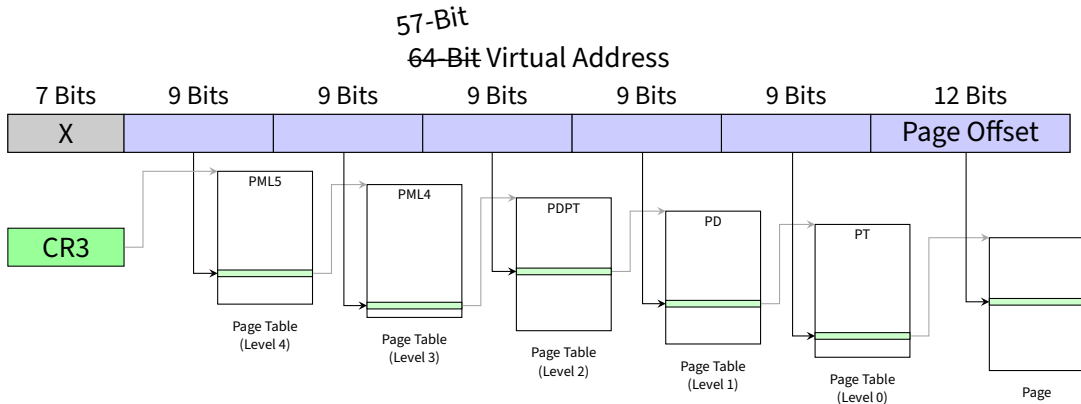
- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level

- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations



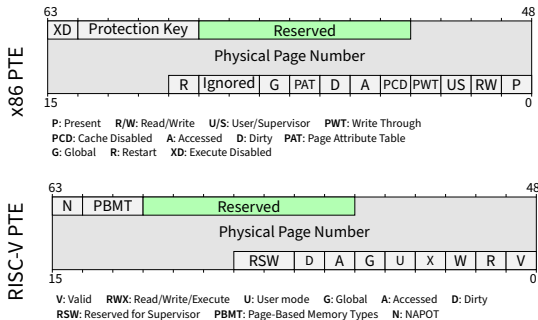
- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level

- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations

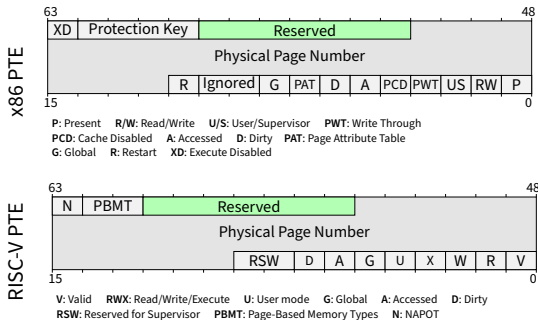


- PTW performs page table walk
- CR3 register points to root PPN
- 9 index bits per level
- 12 index bits in leaf page
- Translation Lookaside Buffer (TLB) caches translations

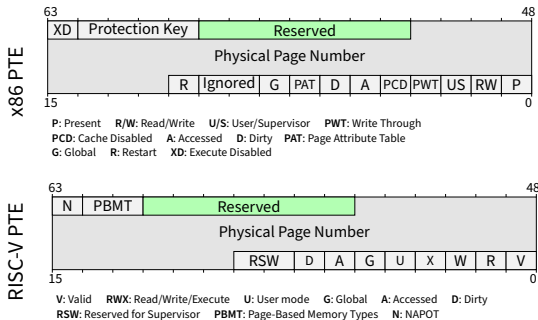
- Page table entries have **spare bits**
- (Security) features require PTE bits
- Number of bits declined in the past
- Adding features requires more bits



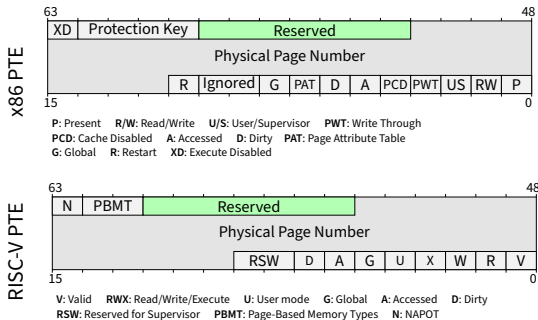
- Page table entries have **spare bits**
- **(Security) features** require PTE bits
- Number of bits declined in the past
- Adding features requires **more bits**



- Page table entries have **spare bits**
- **(Security) features** require PTE bits
- **Number of bits declined** in the past
- Adding features requires **more bits**

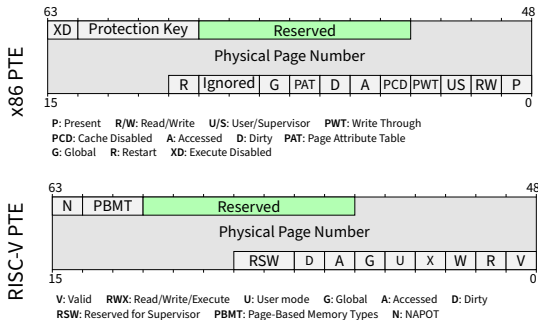


- Page table entries have **spare bits**
- **(Security) features** require PTE bits
- **Number of bits declined** in the past
- Adding features requires **more bits**



Feb. 2025, tech-privileged@lists.riscv.org

- Page table entries have **spare bits**
- **(Security) features** require PTE bits
- **Number of bits declined** in the past
- Adding features requires **more bits**



Approach and Design

- Metadata in PTEs is **essential for security features**
 - Sparse bit count hinders adoption of novel security features
- ↓
- Extend PTEs with dedicated **metadata region**
 - Improve security features, no competition for metadata bits

- Metadata in PTEs is **essential for security features**
- **Sparse bit count hinders adoption** of novel security features



- Extend PTEs with dedicated **metadata region**
- Improve **security features**, no competition for metadata bits

- Metadata in PTEs is **essential for security features**
- **Sparse bit count hinders adoption** of novel security features



- **Extend PTEs with dedicated metadata region**
- Improve security features, no competition for metadata bits

- Metadata in PTEs is **essential for security features**
- **Sparse bit count hinders adoption** of novel security features



- **Extend PTEs** with dedicated **metadata region**
- **Improve security features**, no competition for metadata bits

We identify **four** key parameters

$\mathcal{P}_1$ Metadata Region Size

$\mathcal{P}_2$ Metadata Region Layout

$\mathcal{P}_3$ Affected Table Levels

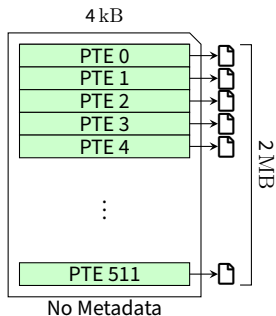
$\mathcal{P}_4$ TLB Entry Size

We identify **four** key parameters

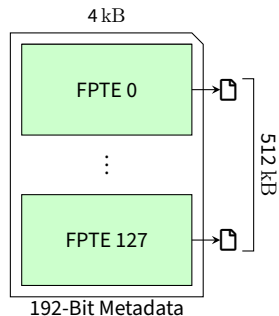
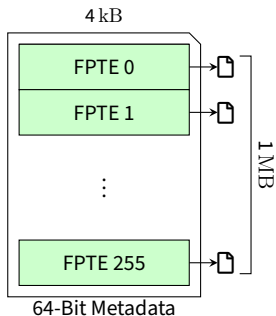
64-Bit or
192-Bit

$$\text{PTE} + \text{MR} \stackrel{!}{=} 2^n$$

$\mathcal{P}_1$ Metadata Region Size



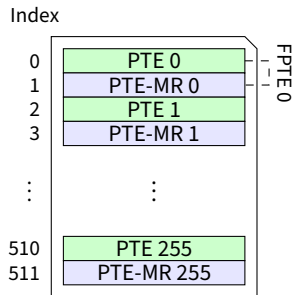
$\mathcal{P}_2$ Metadata Region Layout



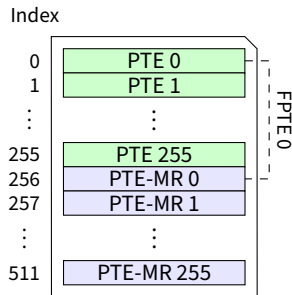
We identify **four** key parameters

Cache-line aligned
or split-page

$\mathcal{P}_1$ Metadata Region Size



$\mathcal{P}_2$ Metadata Region Layout



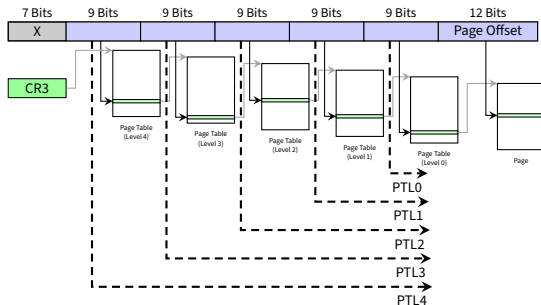
We identify **four** key parameters

$\mathcal{P}_1$ Metadata Region Size

$\mathcal{P}_2$ Metadata Region Layout

$\mathcal{P}_3$ Affected Table Levels

*Last level
or all levels*



We identify **four** key parameters

$\mathcal{P}_1$ Metadata Region Size

$\mathcal{P}_2$ Metadata Region Layout

$\mathcal{P}_3$ Affected Table Levels

$\mathcal{P}_4$ TLB Entry Size

Tradeoff between area and performance



Case Study

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA		# PTE bits
Commercial	Intel CET SHSTK	Access Permission	x86-64		1-bit [†]
	ARM BTI	Legacy Compatibility	ARMv8.5		1-bit
	Intel MPK	Protection Key	x86-64	+	4-bit
	ARM Domains	Protection Key	ARMv7	+	4-bit
	Intel TME-MK	Key Identifier (KeyID)	x86-64	+	15-bit [‡]
Academic	IMIX	Access Permission	x86-64		1-bit
	Donky	Protection Key	RISC-V	+	10-bit
	Multi-Tag	Page-granular Tag	x86-64	+	16-bit
	PT-Guard	Cryptographic MAC	x86-64	+	12-bit
	SecWalk	Redundancy Code	RISC-V	+	25-bit

+ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA		# PTE bits	
Commercial	Intel CET SHSTK	Access Permission	x86-64		1-bit [†]	"Steals" bits from PPN
	ARM BTI	Legacy Compatibility	ARMv8.5		1-bit	
	Intel MPK	Protection Key	x86-64	+	4-bit	
	ARM Domains	Protection Key	ARMv7	+	4-bit	
	Intel TME-MK	Key Identifier (KeyID)	x86-64	+	15-bit [‡]	
Academic	IMIX	Access Permission	x86-64		1-bit	
	Donky	Protection Key	RISC-V	+	10-bit	
	Multi-Tag	Page-granular Tag	x86-64	+	16-bit	
	PT-Guard	Cryptographic MAC	x86-64	+	12-bit	
	SecWalk	Redundancy Code	RISC-V	+	25-bit	

+ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA	# PTE bits	
Commercial	Intel CET SHSTK	Access Permission	x86-64	1-bit [†]	"Steals" bits from PPN
	ARM BTI	Legacy Compatibility	ARMv8.5	1-bit	
	Intel MPK	Protection Key	x86-64	⊕ 4-bit	
	ARM Domains	Protection Key	ARMv7	⊕ 4-bit	
	Intel TME-MK	Key Identifier (KeyID)	x86-64	⊕ 15-bit [‡]	
Academic	IMIX	Access Permission	x86-64	1-bit	Exceeds "reserved" bits
	Donky	Protection Key	RISC-V	⊕ 10-bit	
	Multi-Tag	Page-granular Tag	x86-64	⊕ 16-bit	
	PT-Guard	Cryptographic MAC	x86-64	⊕ 12-bit	
	SecWalk	Redundancy Code	RISC-V	⊕ 25-bit	

⊕ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA	# PTE bits	
Commercial	Intel CET SHSTK	Access Permission	x86-64	1-bit [†]	"Steals" bits from PPN
	ARM BTI	Legacy Compatibility	ARMv8.5	1-bit	
	Intel MPK	Protection Key	x86-64	⊕ 4-bit	
	ARM Domains	Protection Key	ARMv7	⊕ 4-bit	
	Intel TME-MK	Key Identifier (KeyID)	x86-64	⊕ 15-bit [‡]	
Academic	IMIX	Access Permission	x86-64	1-bit	Exceeds "reserved" bits
	Donky	Protection Key	RISC-V	⊕ 10-bit	
	Multi-Tag	Page-granular Tag	x86-64	⊕ 16-bit	
	PT-Guard	Cryptographic MAC	x86-64	⊕ 12-bit	
	SecWalk	Redundancy Code	RISC-V	⊕ 25-bit	

⊕ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA	# PTE bits	
Commercial	Intel CET SHSTK	Access Permission	x86-64	1-bit [†]	"Steals" bits from PPN
	ARM BTI	Legacy Compatibility	ARMv8.5	1-bit	
	Intel MPK	Protection Key	x86-64	⊕ 4-bit	
	ARM Domains	Protection Key	ARMv7	⊕ 4-bit	
	Intel TME-MK	Key Identifier (KeyID)	x86-64	⊕ 15-bit [‡]	
Academic	IMIX	Access Permission	x86-64	1-bit	Exceeds "reserved" bits
	Donky	Protection Key	RISC-V	⊕ 10-bit	
	Multi-Tag	Page-granular Tag	x86-64	⊕ 16-bit	
	PT-Guard	Cryptographic MAC	x86-64	⊕ 12-bit	
	SecWalk	Redundancy Code	RISC-V	⊕ 25-bit	

⊕ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

*$\mathcal{P}_1$ Metadata Region Size
64 Metadata Bits is feasible*

We identify **security features** that use PTE-based metadata

	Mechanism	Application	ISA	# PTE bits	
Commercial	Intel CET SHSTK	Access Permission	x86-64	1-bit [†]	"Steals" bits from PPN
	ARM BTI	Legacy Compatibility	ARMv8.5	1-bit	
	Intel MPK	Protection Key	x86-64	⊕ 4-bit	
	ARM Domains	Protection Key	ARMv7	⊕ 4-bit	
	Intel TME-MK	Key Identifier (KeyID)	x86-64	⊕ 15-bit [‡]	
Academic	IMIX	Access Permission	x86-64	1-bit	Exceeds "reserved" bits
	Donky	Protection Key	RISC-V	⊕ 10-bit	
	Multi-Tag	Page-granular Tag	x86-64	⊕ 16-bit	
	PT-Guard	Cryptographic MAC	x86-64	⊕ 12-bit	
	SecWalk	Redundancy Code	RISC-V	⊕ 25-bit	

⊕ Profits from additional bits † Implicitly encoded ‡ Up to 15-bit

- Existing designs are **hard to implement** due to bit sparsity
- FatPTE allows for **co-existence of multiple features**
- 7/10 designs improve** with increased metadata bits

$\mathcal{P}_1$ Metadata
 64 Metadata
 $\mathcal{P}_3$ No. of levels depends on security features

Evaluation

What are the **performance implications** of our modifications?

Challenges:

- Changing paging requires **hardware changes** (PTW, TLB)
- Changing x86 hardware is **not possible**

What are the **performance implications** of our modifications?

Challenges:

- Changing paging requires **hardware changes** (PTW, TLB)
- Changing x86 hardware is **not possible**

What are the **performance implications** of our modifications?

Challenges:

- Changing paging requires **hardware changes** (PTW, TLB)
- Changing x86 hardware is **not possible**

What are the **performance implications** of our modifications?

Challenges:

- Changing paging requires **hardware changes** (PTW, TLB)
- Changing x86 hardware is **not possible**



We implement FatPTE **using the gem5 simulator**

What are the **performance implications** of our modifications?

Challenges:

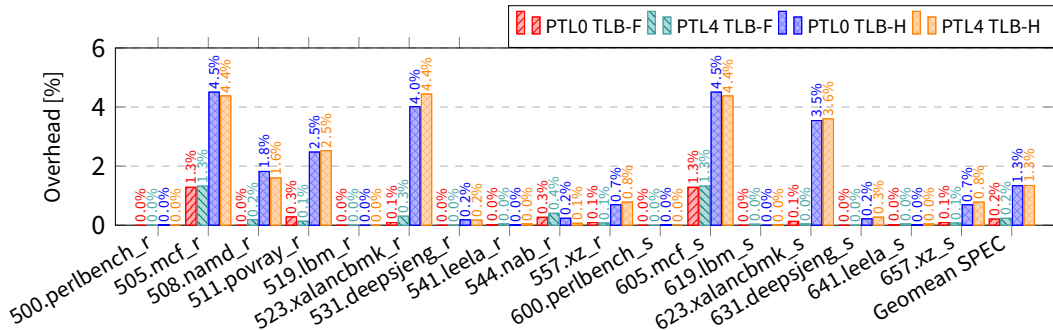
- Changing paging requires **hardware changes** (PTW, TLB)
- Changing x86 hardware is **not possible**



We implement FatPTE **using the gem5 simulator**

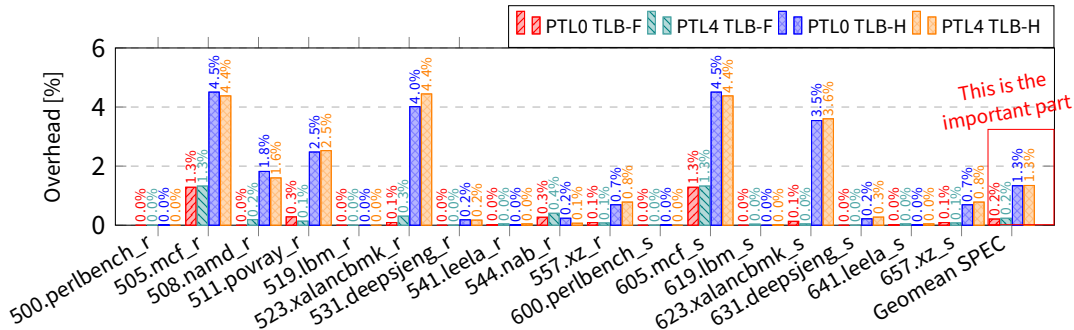
- Evaluate FatPTE in **four configurations**
- **PLT0** vs. **PTL4**
- **TLB-F** vs. **TLB-H**
- **Simpoint**-based simulation
- **O3CPU** configured as Skylake CPU
- **SPEC CPU 2017** workloads

What are the **performance implications** of our modification?



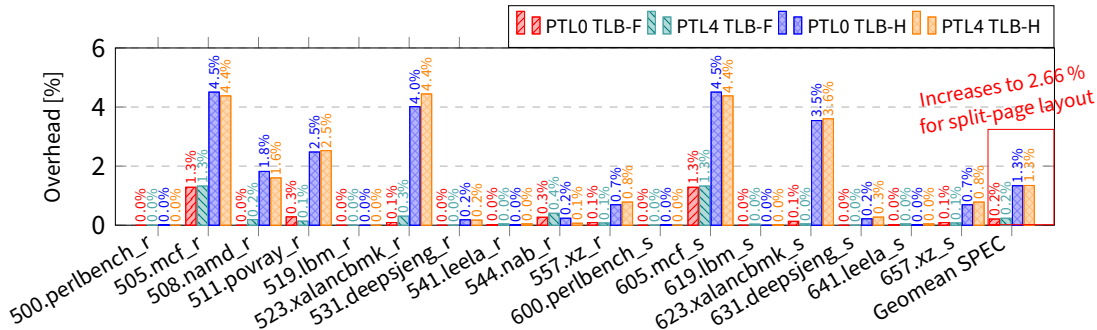
⚠ TLB Size influences overhead strongly

What are the **performance implications** of our modification?



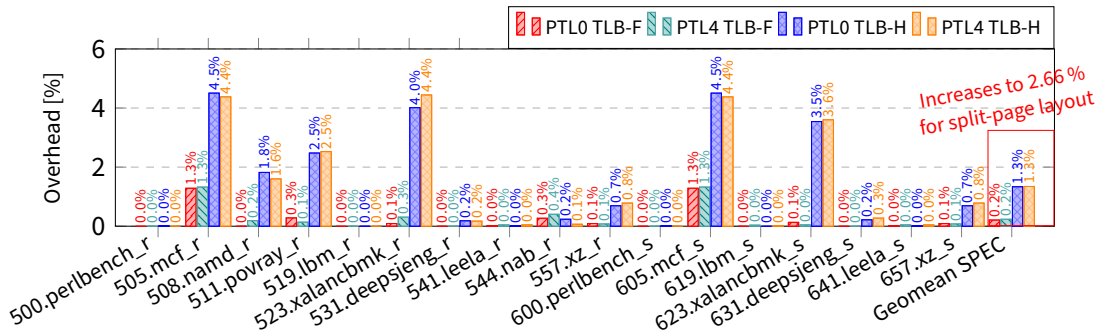
⚠ TLB Size influences overhead strongly

What are the **performance implications** of our modification?



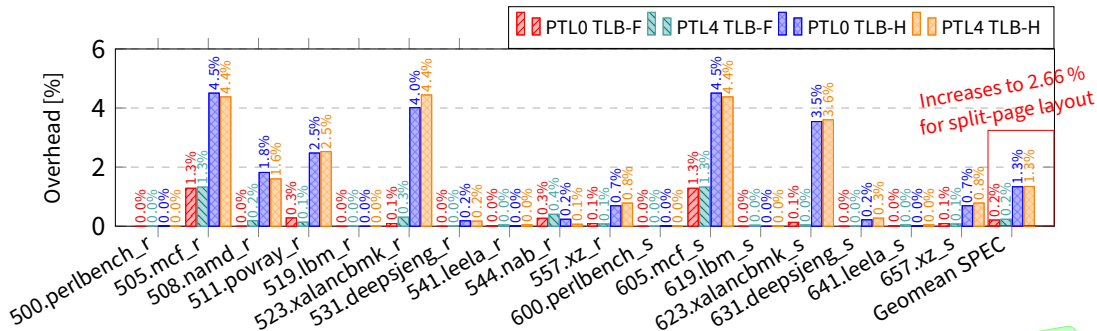
⚠ TLB Size influences overhead strongly

What are the **performance implications** of our modification?



⚠ TLB Size influences overhead strongly

What are the **performance implications** of our modification?



⚠ TLB Size influences overhead strongly

$\mathcal{P}_2$ Cache-Line Aligned Layout is superior

What are the **performance and area implications** of our modifications?

For RISC-V

- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

What are the **performance and area implications** of our modifications?

For RISC-V

- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

What are the **performance and area implications** of our modifications?

For RISC-V

- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

What are the **performance and area implications** of our modifications?

For RISC-V

- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

What are the **performance and area implications** of our modifications?

For RISC-V

- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

**! Significant
area increase**

What are the **performance and area implications** of our modifications?

For RISC-V

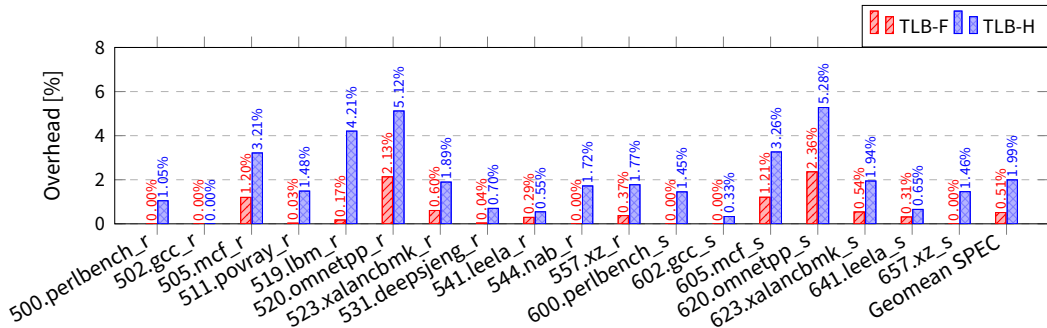
- we implement a **hardware prototype**
- based on the **CVA6 open-source core**

	iTLB	dTLB		LUTs	FFs
Base	64	32	Total	45626	29843
			TLB	7072	8990
TLB-F	64	32	Total	51700 (+13.3%)	36152 (+21.1%)
			TLB	11040 (+56.1%)	15134 (+68.3%)
TLB-H	32	16	Total	45544 (-0.2%)	28519 (-4.4%)
			TLB	3801 (-46.3%)	7566 (-15.8%)

⚠ Significant
area increase

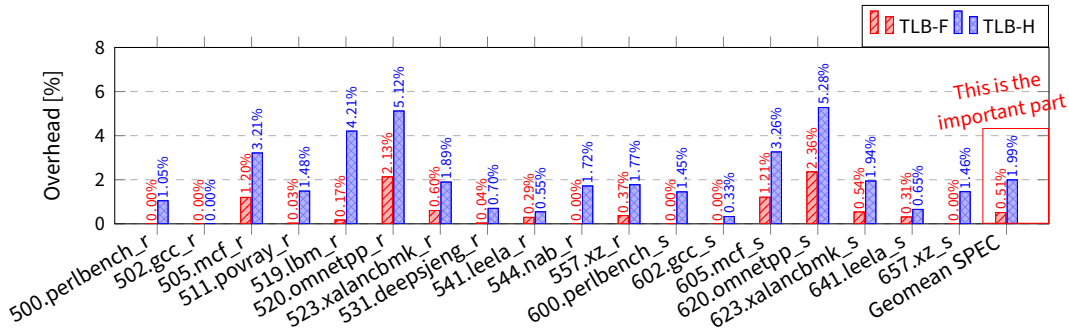
👍 Overall area
does not increase

What are the **performance implications** of our modification?



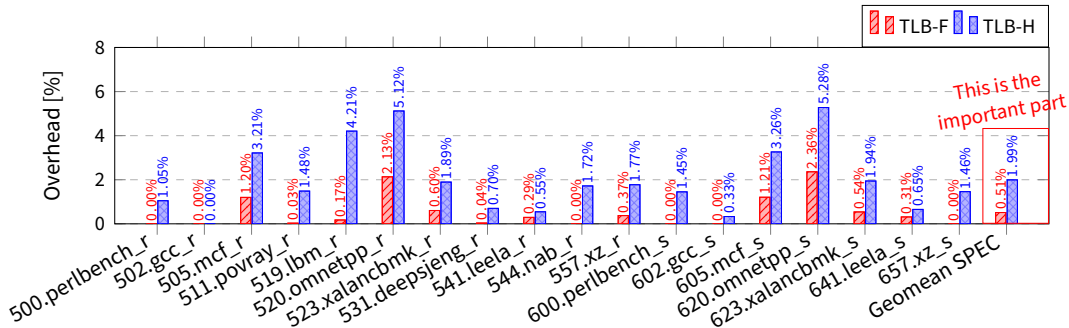
⚠ Even stonger impact than for x86 system
but still a low geomean overhead

What are the **performance implications** of our modification?



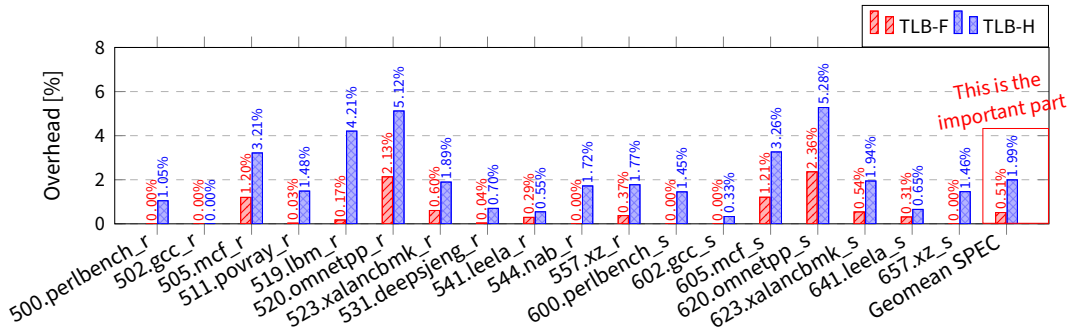
⚠ Even stonger impact than for x86 system
but still a low geomean overhead

What are the **performance implications** of our modification?



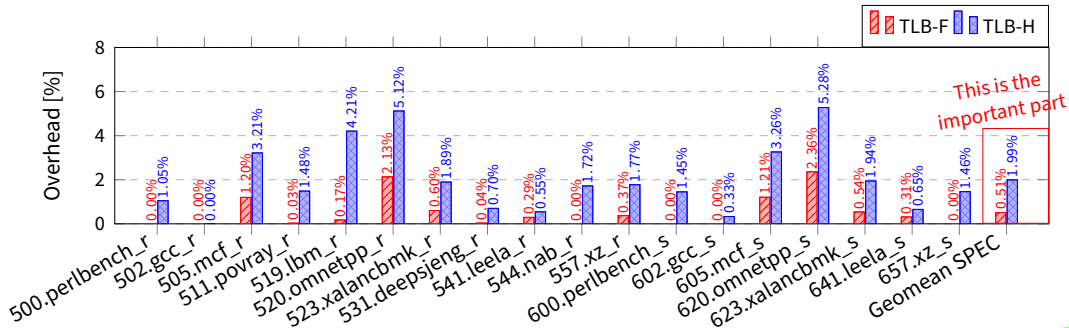
⚠ Even stonger impact than for x86 system
but still a low geomean overhead

What are the **performance implications** of our modification?



⚠ Even stonger impact than for x86 system
but still a low geomean overhead

What are the **performance implications** of our modification?



⚠ Even stonger impact than for x86 system
but still a low geomean overhead

$\mathcal{P}_4$ TLB-H induces
feasible overheads

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🔒 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📄 Influence of different page sizes and synergies with FatPTE
- ⚙️ Comparison with ARMv9 hardware
- 🔗 Combination with segmentation

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🚫 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 Performance overheads are low

Open Questions

- 📄 Influence of different page sizes and synergies with FatPTE
- ⚙️ Comparison with ARMv9 hardware
- 🔗 Combination with segmentation

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🚫 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 Performance overheads are low

Open Questions

- 📏 Influence of different page sizes and synergies with FatPTE
- ⚙️ Comparison with ARMv9 hardware
- 🔗 Combination with segmentation

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🔒 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📏 Influence of different page sizes and synergies with FatPTE
- ⚙️ Comparison with ARMv9 hardware
- 🔗 Combination with segmentation

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🔒 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📏 Influence of different page sizes and synergies with FatPTE
- ⚙️ Comparison with ARMv9 hardware
- 🔗 Combination with segmentation

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🚫 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📄 Influence of different **page sizes** and synergies with FatPTE
- ⚙️ Comparison with **ARMv9** hardware
- 🔗 Combination with **segmentation**

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🚫 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📄 Influence of different **page sizes** and synergies with FatPTE
- ⚙️ Comparison with **ARMv9** hardware
- 🤖 Combination with **segmentation**

Conclusion and Open Questions

- 🔗 Security features often **require page-granular metadata** stored in PTEs
- 🚫 Bit scarcity **hinders adoption and simultaneous use**
- 👍 Increasing the PTE size is a **feasible solution** for that issue
- 📊 **Performance overheads are low**

Open Questions

- 📄 Influence of different **page sizes** and synergies with FatPTE
- ⚙️ Comparison with **ARMv9** hardware
- 🔗 Combination with **segmentation**

FatPTE - Expanding Page Table Entries for Security

Lukas Lamster

¹Graz University of Technology | ²RIVOS, Inc.

ARES 2025

> isec.tugraz.at